

Practical No. 7

[Deadlock Handling]

Sample Data for reference

Process	Allocation				MAX			
	Printer	ROM	Hard Disk	RAM	Printer	ROM	Hard Disk	RAM
Google Drive	0	0	1	4	0	6	5	6
Firefox	0	6	3	2	0	6	5	2
Word Processor	0	0	1	2	0	0	1	2
Excel	1	0	0	0	1	7	5	0
PowerPoint	1	3	5	4	2	3	5	6

Available Resources			
Printer	ROM	Hard Disk	RAM
1	6	2	0

Write a program to :

- Calculate current need of resources by each process.
- Find whether these processes can be executed with available resources. If yes, show the correct order of process execution.

INPUT :

```

~
GNU nano 8.7
#include <stdio.h>

int main() {
    int i, j, k;
    int n = 5; // number of processes
    int m = 4; // number of resources

    int allocation[5][4], max[5][4], need[5][4];
    int available[4];
    int finish[5] = {0};
    int safe[5];

    // Input Allocation Matrix
    printf("Enter Allocation Matrix (5x4):\n");
    for(i = 0; i < n; i++) {
        for(j = 0; j < m; j++) {
            scanf("%d", &allocation[i][j]);
        }
    }

    // Input Max Matrix
    printf("Enter Max Matrix (5x4):\n");
    for(i = 0; i < n; i++) {
        for(j = 0; j < m; j++) {
            scanf("%d", &max[i][j]);
        }
    }

    // Input Available Resources
    printf("Enter Available Resources (4 values):\n");
    for(i = 0; i < m; i++) {
        scanf("%d", &available[i]);
    }

    // Calculate Need Matrix
    for(i = 0; i < n; i++) {
        for(j = 0; j < m; j++) {
            need[i][j] = max[i][j] - allocation[i][j];
        }
    }

    int count = 0;
    while(count < n) {
        int found = 0;
        for(i = 0; i < n; i++) {
            if(finish[i] == 0) {
                int possible = 1;
                for(j = 0; j < m; j++) {
                    if(need[i][j] > available[j]) {
                        possible = 0;
                        break;
                    }
                }
                if(possible) {
                    // Safe state found
                    // ... (rest of the code for safe state)
                }
            }
        }
        if(found == 0) break;
        count++;
    }
}

```

```

        }
    }
    if(possible) {
        for(k = 0; k < m; k++) {
            available[k] += allocation[i][k];
        }
        safe[count] = i;
        count++;
        finish[i] = 1;
        found = 1;
    }
}
if(found == 0) {
    printf("\nSystem is in DeadLock!\n");
    return 0;
}
printf("\nSystem is in Safe State.\nSafe Sequence: ");
for(i = 0; i < n; i++) {
    printf("P%d ", safe[i]);
}
return 0;
}

```

OUTPUT :

```

HP@Hiteshri MSYS ~
$ nano banker.c

HP@Hiteshri MSYS ~
$ gcc banker.c -o banker

HP@Hiteshri MSYS ~
$ ./banker.exe
Enter Allocation Matrix (5x4):
0 0 1 4
0 6 3 2
0 0 1 2
1 0 0 0
1 3 5 4
Enter Max Matrix (5x4):
0 6 5 6
0 6 5 2
0 0 1 2
1 7 5 0
2 3 5 6
Enter Available Resources (4 values):
1 6 2 0

System is in Safe State.
Safe Sequence: P1 P2 P3 P4 P0

```